

Roteiro Prática 4

Samuel Tomazela

2 de outubro de 2026

1 Introdução

Este relatório apresenta a implementação e a simulação dos circuitos desenvolvidos na Prática 4 de Laboratório de Arquitetura e Organização de Computadores I. A prática aborda a implementação de blocos fundamentais de um processador, incluindo um somador completo parametrizável de 16 bits, uma memória de dados de 64 bytes, uma Unidade Lógica e Aritmética (ULA) de 32 bits e um bloco extensor de sinal de 16 para 32 bits.

Os circuitos foram descritos em Verilog HDL e testados por meio de simulações no Quarta Altera Starter FPGA Edition. As formas de onda apresentadas neste relatório têm como objetivo demonstrar o funcionamento dos módulos implementados.

2 Implementação dos circuitos

2.1 Exercício 1: Somador completo de 16 bits

O primeiro exercício consiste na implementação de um somador completo com dois operandos de 16 bits. O circuito é construído a partir de somadores completos de 1 bit, utilizando as construções `generate` e `parameter` da linguagem Verilog. A estrutura permite conectar os somadores de 1 bit em sequência, propagando o carry entre os estágios.

2.1.1 Módulo add16bits

```
1 module add16bits (op1, op2, result);
2
3 parameter SIZE = 16;
4
5 input [SIZE-1:0] op1, op2;
6 output [SIZE:0] result;
7
8 wire [SIZE-1:0] w;
9
10 halfAdder1bit s0(op1[0], op2[0], w[0], result[0]);
11 genvar i;
12 generate
13     for(i = 1; i < SIZE; i = i + 1) begin : gen_loop
14         fulladder1bit s0(op1[i], op2[i], w[i-1], w[i], result[i]);
15     end
16 endgenerate
17
18 assign result[SIZE] = w[SIZE-1];
19
20 endmodule
```

2.1.2 Módulo topLevel

O módulo `topLevel` realiza a conexão do somador de 16 bits aos demais blocos utilizados no projeto, incluindo os displays de 7 segmentos.

```
1 module topLevel(  
2     op1,  
3     op2,  
4     result,  
5     SW,  
6     HEX0,  
7     HEX1  
8 );  
9  
10 parameter SIZE = 16;  
11  
12 input [SIZE-1:0] op1, op2;  
13 output [SIZE:0] result;  
14  
15 input [7:0] SW;  
16 output [6:0] HEX0, HEX1;  
17  
18 add16bits s0(op1, op2, result);  
19 hexDisplay h0(result, HEX0);  
20 hexDisplay h1(result, HEX1);  
21  
22 endmodule
```

2.1.3 Módulo halfAdder1bit

O módulo `halfAdder1bit` implementa um somador de meio bit, produzindo a soma e o carry de saída a partir de dois bits de entrada.

```
1 module halfAdder1bit(a, b, c_out, s);  
2  
3 input a, b;  
4 output s, c_out;  
5  
6 assign s = a ^ b;  
7 assign c_out = a & b;  
8  
9 endmodule
```

2.1.4 Módulo fulladder1bit

O módulo `fulladder1bit` implementa um somador completo de 1 bit, recebendo dois operandos e um carry de entrada e produzindo a soma e o carry de saída.

```
1 module fulladder1bit(a, b, c_in, c_out, s);  
2  
3 input a, b, c_in;  
4 output s, c_out;  
5  
6 assign s = a ^ b ^ c_in;  
7 assign c_out = (~a & b & c_in) | (a & ~b & c_in) | (a & b);  
8  
9 endmodule
```

2.2 Exercício 2: Memória de dados

O segundo exercício consiste na implementação parametrizável de uma memória de dados com capacidade de armazenamento de 64 bytes. Cada posição da memória armazena um byte de 8 bits, enquanto cada palavra utilizada na implementação possui 16 bits e, portanto, ocupa dois bytes.

A lógica de escrita é síncrona, ocorrendo na borda de subida do clock quando `memWrite` está habilitado. A leitura é combinacional e ocorre quando `memRead` está habilitado.

2.2.1 Módulo `memoryModule`

```
1 module memoryModule
2 (
3     address,
4     memWrite,
5     memRead,
6     clock,
7     writeData,
8     readData
9 );
10
11 input [5:0] address;
12 input memWrite, memRead, clock;
13 input [15:0] writeData;
14 output [15:0] readData;
15 reg [7:0] memory [0:63];
16
17 assign readData = memRead ? {memory[address + 1], memory[address]} :
18     16'b0;
19
20 always @(posedge clock) begin
21     if(memWrite && address < 63) begin
22         memory[address] = writeData[7:0];
23         memory[address + 1] = writeData[15:8];
24     end
25 end
26
27 endmodule
```

2.2.2 Módulo `tb_memoryModule`

O testbench foi utilizado para verificar a operação de escrita e posterior leitura de uma palavra de 16 bits na memória.

```
1 module tb_memoryModule;
2
3     reg [4:0] address;
4     reg memWrite, memRead, clock;
5     reg [15:0] writeData;
6     wire [15:0] readData;
7
8     memoryModule m(
9         .address(address),
10        .memWrite(memWrite),
11        .memRead(memRead),
12        .clock(clock),
13        .writeData(writeData),
14        .readData(readData)
15    );
16
```

```

17 // Gera clock: 0 -> 1 -> 0 -> 1...
18 always #5 clock = ~clock;
19
20 initial begin
21     $monitor("time=%0t | readData=%h | address=%d | memWrite=%b | memRead
22             =%b",
23             $time, readData, address, memWrite, memRead);
24
25     clock = 0;
26
27     // Escrever CA no endereço 0
28     address = 5'b0;
29     memWrite = 1;
30     memRead = 0;
31     writeData = 16'h00CA;
32
33     #10;
34
35     // Agora ler o endereço 0
36     memWrite = 0;
37     memRead = 1;
38
39     #10;
40     $finish;
41 end
42
43 endmodule

```

2.3 Exercício 3: Unidade Lógica e Aritmética (ULA)

O terceiro exercício consiste na implementação de uma Unidade Lógica e Aritmética (ULA) de 32 bits. Conforme especificado na prática, a ULA deve realizar cinco operações: AND, OR, soma, subtração e comparação menor que (SLT).

As operações são selecionadas por meio do sinal de controle `aluOp`. A subtração é realizada utilizando o complemento de dois do segundo operando. A saída `zero` indica quando o resultado da operação é igual a zero, enquanto a saída `overflow` é utilizada para indicar ocorrência de estouro nas operações aritméticas.

2.3.1 Módulo ULAModule

```

1 module ULAModule(
2     op1,
3     op2,
4     aluOp,
5     zero,
6     resultado,
7     overflow
8 );
9
10 input [31:0] op1, op2;
11 input [2:0] aluOp;
12 output reg [31:0] resultado;
13 output reg overflow, zero;
14
15 reg [31:0] temp;
16
17
18 always @(*) begin

```

```

19         case(aluOp)
20             3'b000: resultado = op1 & op2; //and
21             3'b001: resultado = op1 | op2; //or
22             3'b010:
23                 begin
24                     resultado = op1 + op2;
25                     overflow = op1[31] + op2[31];
26                 end
27
28             3'b110: resultado = op1 + (~op2 + 1);
29             3'b111:
30                 begin
31                     temp = op1 + (~op2 + 1);
32                     resultado = 32'b0;
33                     if(temp < 0)
34                         resultado[31] = 1;
35                 end
36
37         endcase
38
39         zero = (resultado == 32'b0);
40     end
41 endmodule
42

```

2.3.2 Módulo tb_ULAModule

O testbench da ULA foi utilizado para aplicar diferentes combinações de operandos e códigos de operação, permitindo observar os resultados produzidos para as operações lógicas, aritméticas e de comparação.

```

1 module tb_ULAModule;
2     // Testbench da ULA
3 endmodule

```

2.4 Exercício 4: Extensão de sinal

O quarto exercício consiste na implementação de um bloco capaz de receber um sinal de 16 bits e apresentar o mesmo valor em uma saída de 32 bits. A extensão é realizada acrescentando os bits necessários à representação do valor de entrada.

2.4.1 Módulo ExtensorDeSinal

```

1 module ExtensorDeSinal
2 (
3     a,
4     b
5 );
6
7     input  [15:0] a;
8     output [31:0] b;
9
10    assign b = {16'b0, a};
11
12 endmodule

```

2.4.2 Módulo tb_ExtensorDeSinal

```
1 module tb_ExtensorDeSinal;  
2     reg [15:0] a = 16'b1;  
3     wire [31:0] b;  
4  
5     ExtensorDeSinal m(a, b);  
6 endmodule
```

3 Provação por simulação

As simulações foram realizadas no Questa Altera Starter FPGA Edition. As formas de onda a seguir apresentam os sinais de entrada e saída dos circuitos e permitem verificar o comportamento dos módulos implementados.

3.1 Exercício 1: Somador completo de 16 bits

A simulação do somador de 16 bits permite observar os operandos de entrada e o resultado da soma, incluindo o bit adicional correspondente ao carry de saída. A forma de onda demonstra o funcionamento da cadeia de somadores de 1 bit utilizada na construção do circuito.

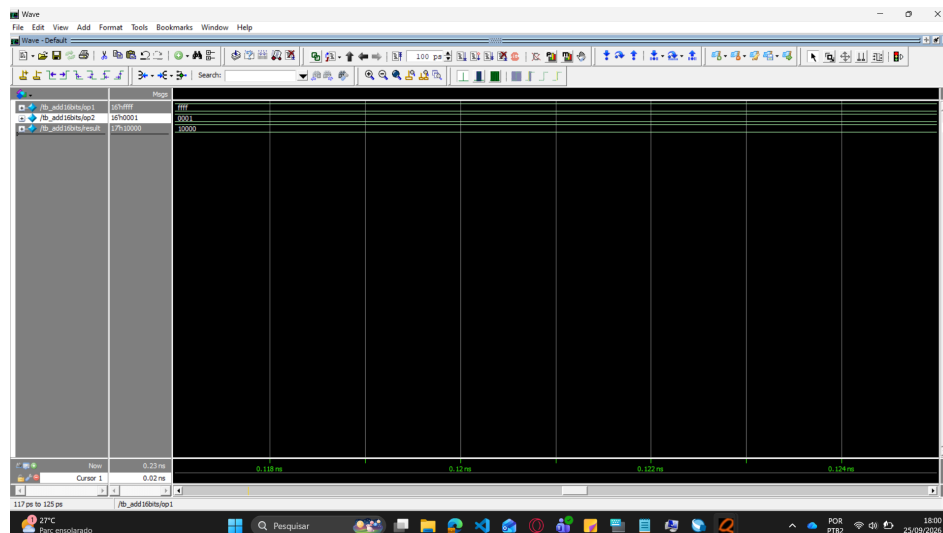


Figura 1: Forma de onda da simulação do somador completo de 16 bits.

3.2 Exercício 2: Memória de dados

A simulação da memória demonstra inicialmente uma operação de escrita, com `memWrite` habilitado e o dado fornecido na entrada `writeData`. Posteriormente, `memWrite` é desabilitado e `memRead` é habilitado, permitindo observar na saída `readData` o valor armazenado no endereço selecionado.

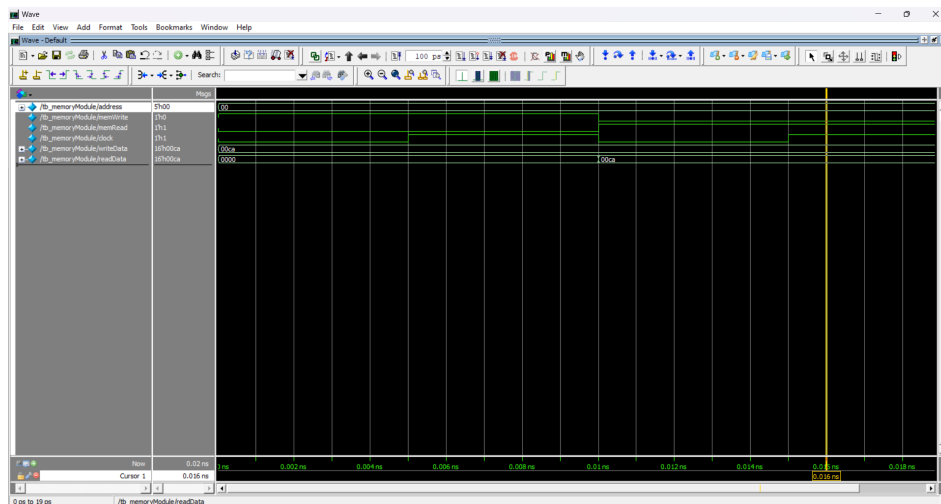
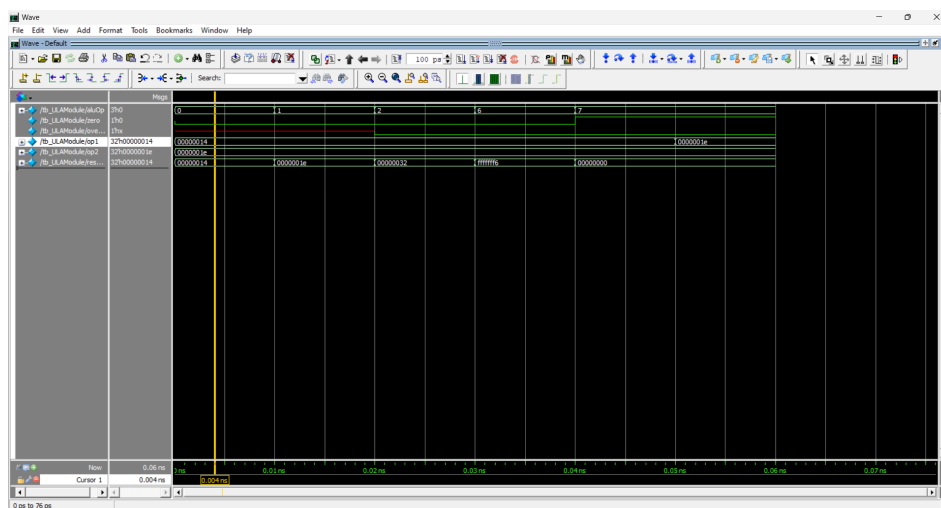


Figura 2: Forma de onda da simulação da memória de dados.

3.3 Exercício 3: ULA

A simulação da ULA apresenta diferentes valores para o sinal de controle `aluOp` e para os operandos de entrada. Dessa forma, é possível observar os resultados das operações AND, OR, soma, subtração e comparação menor que, além do comportamento da saída `zero`.



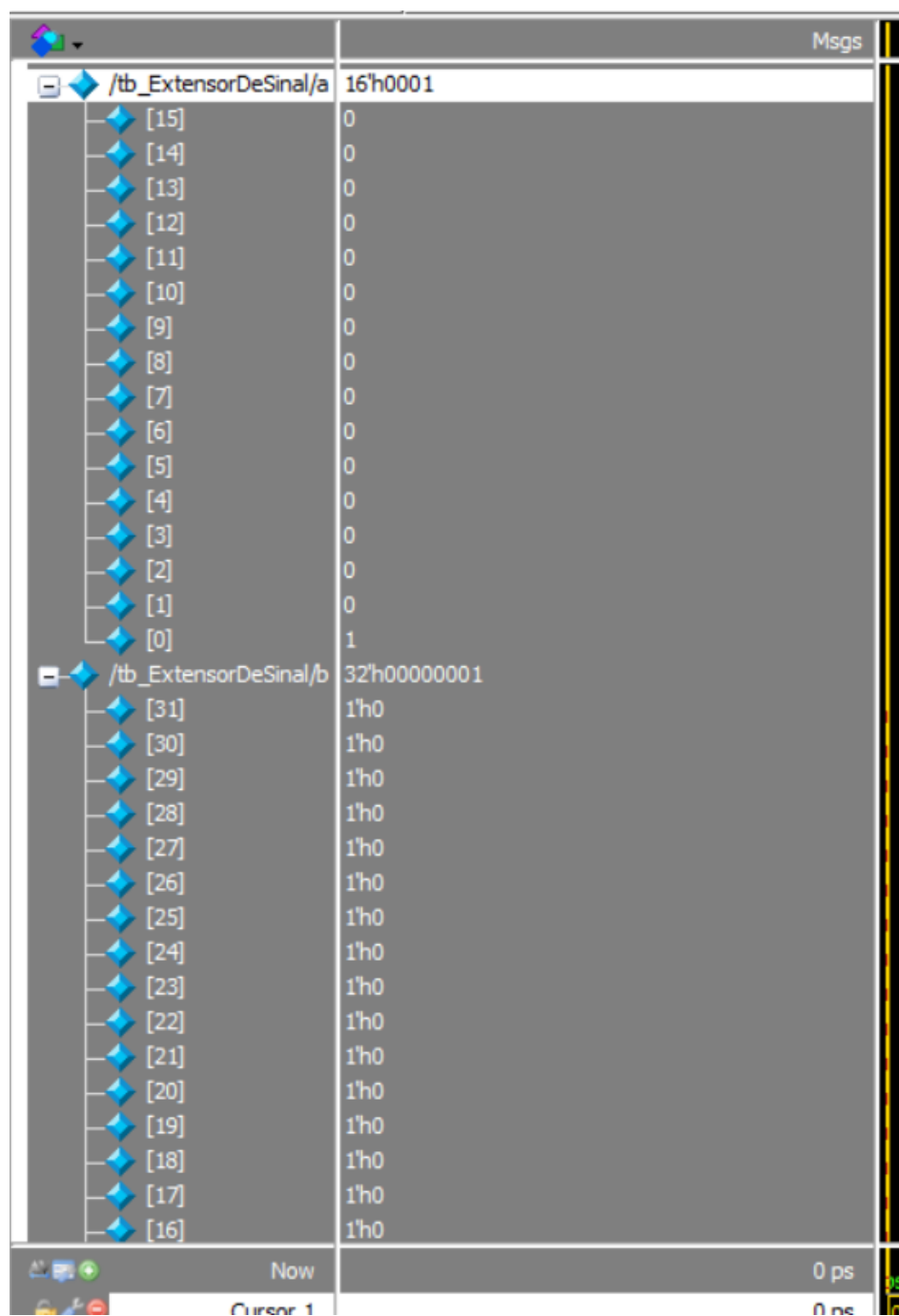


Figura 4: Primeira forma de onda da simulação do extensor de sinal.

[0]	1
/tb_ExtensorDeSinal/b	32'h00000001
[31]	1'h0
[30]	1'h0
[29]	1'h0
[28]	1'h0
[27]	1'h0
[26]	1'h0
[25]	1'h0
[24]	1'h0
[23]	1'h0
[22]	1'h0
[21]	1'h0
[20]	1'h0
[19]	1'h0
[18]	1'h0
[17]	1'h0
[16]	1'h0
[15]	1'h0
[14]	1'h0
[13]	1'h0
[12]	1'h0
[11]	1'h0
[10]	1'h0
[9]	1'h0
[8]	1'h0
[7]	1'h0
[6]	1'h0
[5]	1'h0
[4]	1'h0
[3]	1'h0
[2]	1'h0
[1]	1'h0
[0]	1'h1

Figura 5: Segunda forma de onda da simulação do extensor de sinal.